

Trong giai đoạn trước (Bài 07), quá trình phân tích và đánh giá thực nghiệm hệ thống Microservices thương mại điện tử đã giúp nhóm nhận diện được một số điểm nghẽn nghiêm trọng trong kiến trúc phần mềm. Cụ thể, sự phụ thuộc chặt chẽ (High Coupling) thông qua giao tiếp đồng bộ giữa các dịch vụ đã làm giảm khả năng mở rộng của hệ thống. Bên cạnh đó, sự thiếu hụt các cơ chế tự phục hồi (Circuit Breaker), lỗi hỏng trong xác thực liên dịch vụ (Inter-service Authentication), cùng những vi phạm về nguyên lý thiết kế SOLID (God Class, Logic Duplication) đã làm tăng rủi ro vận hành và gây khó khăn cho việc bảo trì.

Để giải quyết các bài toán kiến trúc phức tạp nêu trên, thay vì tiến hành lập trình thủ công truyền thống tốn nhiều thời gian, nhóm quyết định ứng dụng Trí tuệ Nhân tạo tạo sinh (GenAI) vào quy trình tái cấu trúc phần mềm. Báo cáo Bài 08 này trình bày chi tiết hành trình nâng cấp hệ thống Microservices thông qua việc sử dụng các công cụ GenAI tích hợp trong IDE (như GitHub Copilot Chat, Antigravity). Báo cáo sẽ minh chứng rằng, khi người kỹ sư làm chủ được ngữ cảnh và tư duy thiết kế, GenAI không chỉ là công cụ hỗ trợ gõ mã nhanh (code completion) mà còn đóng vai trò như một "chuyên gia lập trình cặp" (Pair Programmer) đắc lực, giúp hiện thực hóa các giải pháp kiến trúc nâng cao một cách chính xác và hiệu quả.

Chương 1. PHƯƠNG PHÁP THỰC HIỆN: ỨNG DỤNG PROMPT ENGINEERING TRONG TÁI CẤU TRÚC

Thay vì sử dụng các câu lệnh (prompt) đơn giản và ngẫu hứng (thường dẫn đến hiện tượng AI sinh mã nguồn sai lệch hoặc không phù hợp với tổng thể kiến trúc), nhóm đã áp dụng các kỹ thuật Prompt Engineering nâng cao (Advanced Prompt Engineering) xuyên suốt quá trình thực hiện. Phương pháp tiếp cận được cá nhân hóa dựa trên nguyên tắc **5S (Structured, Surrounding context, Single task, Specific, Short)**, bao gồm các kỹ thuật cốt lõi sau:

1. Định hình Vai trò và Ngữ cảnh (Role Prompting & Context Scaffolding) Trước khi yêu cầu GenAI sinh mã nguồn, nhóm luôn khởi tạo bối cảnh bằng cách ép AI đóng vai trò là một chuyên gia (Senior Engineer) và cung cấp đầy đủ thông tin về nền tảng công nghệ đang sử dụng (như Spring Boot 3.2, Spring Cloud Gateway, Java 17). Điều này giúp thu hẹp phạm vi tìm kiếm của mô hình ngôn ngữ (LLM), loại bỏ việc AI đề xuất các thư viện lỗi thời (deprecated).

2. Ép suy luận từng bước (Chain-of-Thought Prompting) Đối với các nghiệp vụ phức tạp như thiết lập cơ chế xác thực JWT hay phân luồng Event-Driven với Kafka, nhóm áp dụng kỹ thuật Chain-of-Thought. GenAI được yêu cầu phải phác thảo tư duy logic (step-by-step logic) và vẽ ra luồng dữ liệu trước khi trực tiếp viết mã nguồn. Việc này cho phép nhóm kiểm

duyet giải pháp kiến trúc của AI ở mức độ thiết kế hệ thống (System Design) trước khi đi vào hiện thực hóa.

3. Thiết lập Ràng buộc nghiêm ngặt (Constraint Prompting) Để giải quyết tình trạng vi phạm nguyên lý SOLID đã nêu ở Bài 07, các prompt được bổ sung các ràng buộc (Constraints) cực kỳ khắt khe. AI bị buộc phải tuân thủ nguyên lý Đơn nhiệm (Single Responsibility Principle) bằng cách bóc tách logic ánh xạ dữ liệu (Mapping) ra khỏi logic nghiệp vụ (Service), hoặc phải áp dụng một định dạng dữ liệu (JSON format) thống nhất trên toàn hệ thống.

4. Tinh chỉnh lặp đi lặp lại (Iterative Refinement) Quá trình Vibe Coding không diễn ra trong một bước duy nhất. Nhóm tận dụng các thông báo lỗi hoặc rà soát mã nguồn do AI sinh ra ở lần đầu tiên để tiếp tục prompt yêu cầu tinh chỉnh (refactor), bắt các ngoại lệ (Exception) bị thiếu, và bổ sung các kịch bản kiểm thử (Test cases) nhằm đạt được chất lượng mã nguồn tối ưu nhất.

Bằng việc kết hợp hệ thống phương pháp luận này, toàn bộ quá trình giao tiếp thực tế với GenAI (được minh chứng qua các hình ảnh chụp từ công cụ Chat trong IDE ở các phần tiếp theo) đã giúp nhóm giải quyết trọn vẹn các thách thức kiến trúc đặt ra từ Bài 07.

CHƯƠNG 2: QUÁ TRÌNH ỨNG DỤNG GENAI TỐI ƯU HỆ THỐNG

2.1 Giải quyết lỗ hổng bảo mật với JWT Authentication tại API Gateway

Vấn đề: Theo như phân tích từ Bài 07, hệ thống đang gặp thách thức về an ninh mạng khi thiếu hụt cơ chế xác thực tập trung. Các yêu cầu đi qua API Gateway hiện chưa được kiểm soát chặt chẽ bằng JWT (JSON Web Token), tạo ra lỗ hổng bảo mật nghiêm trọng khi các dịch vụ nội bộ có thể bị truy cập trái phép.

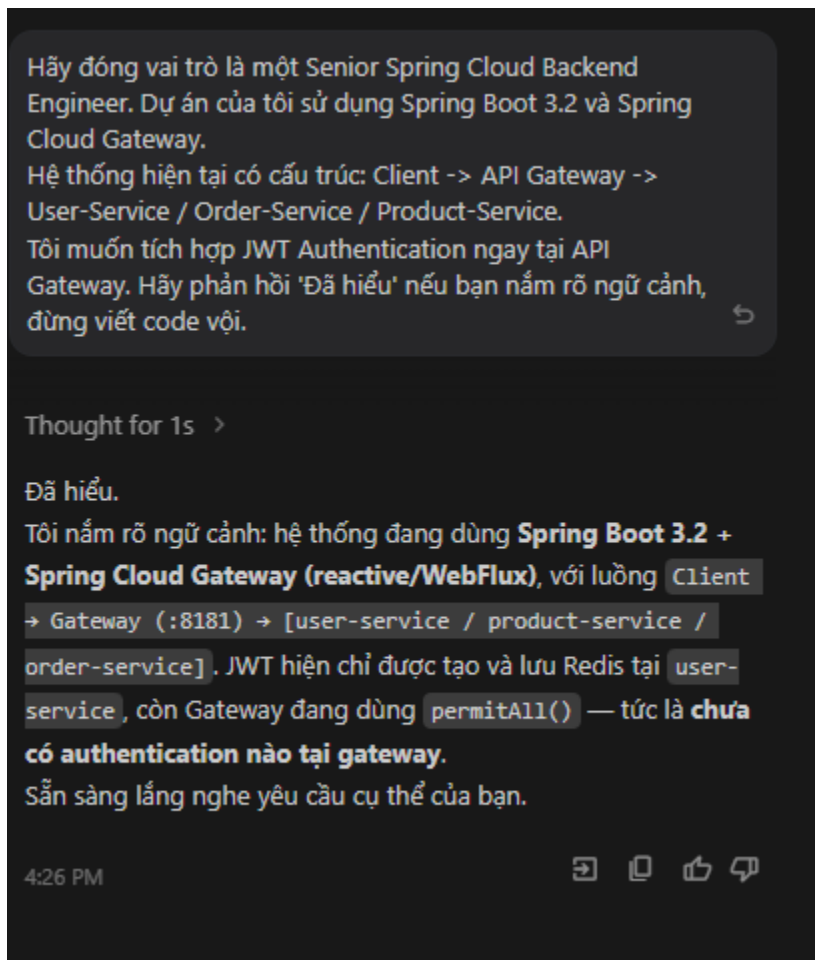
Quá trình ứng dụng GenAI: Nhóm đã sử dụng GitHub Copilot Chat để thiết kế và triển khai JwtAuthenticationFilter trực tiếp tại API Gateway. Thay vì yêu cầu AI sinh mã nguồn ngay lập tức, nhóm đã áp dụng phương pháp **Prompt Engineering** với các kỹ thuật:

- **Role Prompting & Context Scaffolding:** Yêu cầu AI đóng vai một "Senior Spring Cloud Backend Engineer", đồng thời cung cấp đầy đủ bối cảnh về hệ thống (Spring Boot 3.2, Spring Cloud Gateway, kiến trúc hiện tại) để AI nắm bắt trước khi thực hiện nhiệm vụ.

- **Chain of Thought (Ép suy luận từng bước):** Yêu cầu AI vạch ra "step-by-step logic" để thiết kế Filter, từ việc chọn đúng abstraction (AbstractGatewayFilterFactory), đến cách trích xuất token, thiết lập whitelist các route public, và cách xử lý khi token lỗi.

Dưới đây là một số hình ảnh minh chứng quá trình prompt với AI:

Prompt 1: Thiết lập vai trò chuyên gia và cung cấp ngữ cảnh hệ thống cho AI.



Prompt 2: AI áp dụng Chain-of-Thought, trình bày chi tiết logic từng bước để xây dựng JWT Filter.

Tốt. Trước khi viết code, hãy liệt kê step-by-step logic cần thiết để tạo một GatewayFilterFactory chuyên đọc header Authorization, trích xuất token JWT, kiểm tra tính hợp lệ và định tuyến request. Hãy suy nghĩ từng bước một

Thought for 21s >

Đây là phân tích logic từng bước, bao gồm cả các **quyết định thiết kế quan trọng** cần cân nhắc trước khi viết code:

Step-by-step Logic

Step 1 — Chọn đúng abstraction

Dùng `AbstractGatewayFilterFactory` thay vì `WebFilter` vì:

- Cho phép apply **per-route** (chỉ bảo vệ route cần thiết, whitelist dễ hơn)
- Có inner `Config` class để cấu hình linh hoạt qua `application.properties`
- Tích hợp tự nhiên với DSL routing của Spring Cloud Gateway

Step 2 — Intercept request (reactive)

Gateway là **WebFlux (non-blocking)**, nên toàn bộ logic phải trả về `Mono<Void>` :

```
GatewayFilter.apply() → (ServerWebExchange, GatewayFilterChain) → Mono<Void>
```

Lấy header từ: `exchange.getRequest().getHeaders().getFirst(HttpHeaders.AUTHORIZATION)`

Step 3 — Whitelist các route public

Trước khi check token, phải bỏ qua các endpoint **không cần auth**:

- `POST /v1/api/user/login`
- `POST /v1/api/user/register`
- `GET /v1/api/products/**` (tùy *business logic*)

Nếu path nằm trong whitelist → gọi `chain.filter(exchange)` luôn, không kiểm tra.

Step 4 — Kiểm tra sự tồn tại của header

```
Header null hoặc không bắt đầu bằng "Bearer "  
→ trả về 401 ngay, KHÔNG forward request
```

Step 5 — Trích xuất JWT string

"Bearer eyJhbGciOiJ..." → strip "Bearer " → lấy raw token string

Step 6 — ⚠️ Quyết định thiết kế quan trọng: Validate ở đâu?

Có 2 lựa chọn:

	Option A: Validate cục bộ tại Gateway	Option B: Gọi user-service/Redis
Cách làm	Parse + verify signature bằng secret key	HTTP call đến user-service hoặc query Redis trực tiếp
Hiệu năng	✅ Nhanh, không network call	❌ Thêm latency mỗi request
Độc lập	✅ Gateway không phụ thuộc user-service	❌ user-service phải up
Revoke token	❌ Không biết token đã logout chưa	✅ Kiểm tra được Redis blacklist

→ Khuyến nghị: Option A (local validation) + thêm cơ chế Redis check chỉ khi cần revoke.

⚠️ Vấn đề hiện tại của project: `JWTUtil` đang dùng `Keys.secretKeyFor(HS256)` — sinh ngẫu nhiên mỗi lần restart. Gateway không thể chia sẻ secret này. Phải đổi sang fixed secret key trong `application.properties` trước khi làm bước này.

Step 7 — Parse và verify JWT (local)

Trong `Mono.fromCallable()` (tránh block event loop):

1. Khởi tạo `JwtParser` với **fixed secret key** (shared giữa user-service và gateway)
2. `parser.parseClaimsJws(token)` → nếu throw `ExpiredJwtException` → 401 "Token expired"
3. Nếu throw `JwtException` khác → 401 "Invalid token"
4. Nếu thành công → lấy `Claims` object

Tốt. Trước khi viết code, hãy liệt kê step-by-step logic cần thiết để tạo một GatewayFilterFactory chuyên đọc header Authorization, trích xuất token JWT, kiểm tra tính hợp lệ và định tuyến request. Hãy suy nghĩ từng bước một

Step 8 — Extract claims và mutate request

Lấy thông tin từ claims và **đính kèm vào request headers** trước khi forward:

```
X-User-Id    : claims.get("id")
X-User-Email : claims.getSubject()
X-User-Role  : claims.get("role") + isAdmin flag
```

Dùng `exchange.getRequest().mutate().header(...).build()` → tạo request mới immutably.

Step 9 — Forward hoặc Reject

```
Token hợp lệ → chain.filter(mutatedExchange) [forward downstream]
Token lỗi    → response 401 với JSON error body
```

Step 10 — Apply filter vào routes cần bảo vệ

Trong `application.properties`, chỉ apply filter cho các route cần bảo vệ:

```
# Route public (không filter)
/v1/api/user/login
/v1/api/user/register

# Route protected (có filter)
/v1/api/order/**
/v1/api/payment/**
```

Dựa trên logic vừa phân tích, hãy viết class `JwtAuthenticationFilter.java`.
Ràng buộc (Constraints):

Tránh xa các class đã bị deprecated như `WebSecurityConfigurerAdapter`.

Sử dụng class `jjwt-api` phiên bản mới nhất (0.12.x) cho việc parse token.

Tách logic kiểm tra token ra một class tiện ích riêng `JwtUtil.java` (Single Responsibility Principle).

Nếu token không hợp lệ, trả về HTTP Status 401 (Unauthorized) một cách gọn gàng.

Worked for 5m >

Rebuild cả 2 service:

```
bash
```

```
docker compose up -d --build user-service api-gateway
```

Tóm tắt thay đổi theo từng ràng buộc

Ràng buộc	Cách xử lý
Không dùng <code>WebSecurityConfigurerAdapter</code>	Dùng <code>GlobalFilter + Ordered</code> — hoàn toàn độc lập với Spring Security
JWT 0.12.x API	<code>verifyWith()</code> · <code>parseSignedClaims()</code> · <code>getPayload()</code> · <code>signWith(key)</code> · <code>subject()</code> · <code>issuedAt()</code> · <code>expiration()</code>
SRP — tách JwtUtil riêng	<code>JwtUtil.java</code> chỉ chứa logic parse/verify · <code>JwtAuthenticationFilter.java</code> chỉ chứa filter flow
401 gọn gàng	<code>onErrorResume</code> phân loại <code>ExpiredJwtException</code> vs <code>JwtException</code> → JSON <code>{"status":401,...}</code>

2.2. Gỡ bỏ High Coupling bằng kiến trúc Event-Driven (Apache Kafka)

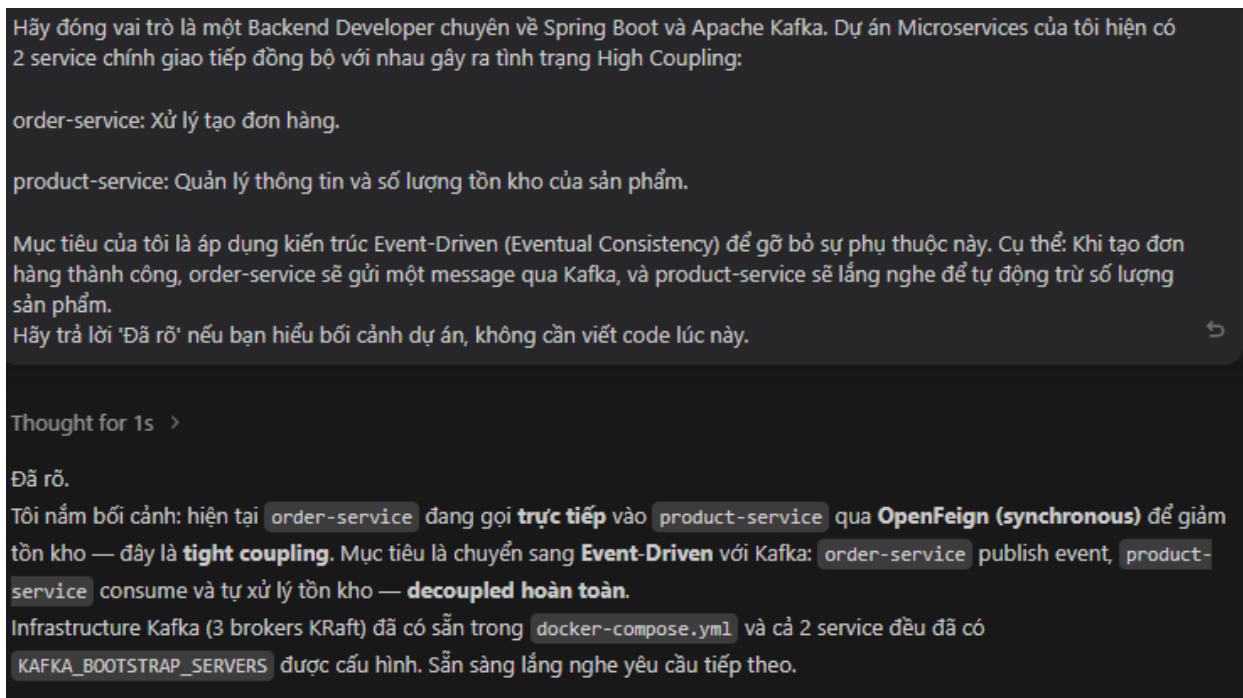
1. Vấn đề hiện tại: Theo đánh giá kiến trúc tại Bài 07, điểm nghẽn lớn nhất của hệ thống là sự phụ thuộc chặt chẽ (High Coupling) qua giao tiếp đồng bộ giữa `order-service` và `product-service`. Việc `order-service` sử dụng `FeignClient` để gọi trực tiếp API trữ tồn kho khiến toàn bộ luồng tác vụ đặt hàng có nguy cơ thất bại (Cascading Failure) nếu `product-service` gặp sự cố hoặc mạng lưới có độ trễ cao.

2. Quá trình ứng dụng GenAI để tái cấu trúc: Để giải quyết triệt để vấn đề này, nhóm quyết định chuyển đổi sang mô hình hướng sự kiện (Event-Driven) với Apache Kafka, chấp nhận tính nhất quán cuối cùng (Eventual Consistency). Thay vì viết code thủ công, nhóm đã điều

hướng AI thực hiện việc tái cấu trúc thông qua một chuỗi prompt (Prompt Chain) tuân thủ nguyên tắc 5S:

- **Bước 1: Thiết lập bối cảnh kiến trúc (Context Scaffolding)** Thay vì yêu cầu viết code ngay, nhóm bắt đầu bằng việc thiết lập vai trò chuyên gia (Backend Developer & Apache Kafka) cho AI. Nhóm mô tả rõ hiện trạng (đồng bộ qua OpenFeign) và mục tiêu (bất đồng bộ qua Kafka) để AI nắm bắt toàn cục. AI đã phản hồi "Đã rõ" và xác nhận đã hiểu bối cảnh hạ tầng Kafka hiện có.

Hình minh họa: Thiết lập bối cảnh kiến trúc cho AI



Bước 2: Xây dựng Event và Producer với các ràng buộc khắt khe (Constraint Prompting)

Tại order-service, nhóm yêu cầu AI tạo cấu trúc dữ liệu OrderPlacedEvent và class OrderEventProducer. Ràng buộc (Constraint) được đưa ra rất rõ ràng: phải sử dụng Spring Kafka và serialize dữ liệu dạng JSON. Không chỉ sinh ra mã nguồn chính xác, AI còn chủ động giải thích các quyết định thiết kế quan trọng (ví dụ: dùng orderId làm Key để đảm bảo thứ tự message vào cùng một partition).

Hình minh họa: Prompt tạo Producer và các quyết định thiết kế do AI tổng kết

Tốt. Bây giờ chúng ta sẽ làm việc trên order-service.
CONTEXT: Tôi có class OrderRequest chứa orderId, productId, và quantity.
TASK:
Tạo một class OrderPlacedEvent (chứa orderId, productId, quantity) dùng để làm payload gửi qua Kafka.

Viết class OrderEventProducer sử dụng KafkaTemplate<String, OrderPlacedEvent> để publish event này vào topic có tên là order-placed-topic.

CONSTRAINTS:
Sử dụng Spring Kafka.
Đảm bảo cấu hình Serialization gửi dữ liệu dưới dạng JSON.

Tổng kết những gì đã tạo

Cấu trúc files mới

```
order-service/
├── event/
│   └── OrderPlacedEvent.java      ← NEW: Kafka message payload
├── config/
│   └── KafkaProducerConfig.java  ← UPDATED: thêm Producer 2 (JSON)
└── producer/
    └── OrderEventProducer.java    ← UPDATED: thêm publishOrderPlaced()
```

Điểm thiết kế quan trọng

Quyết định	Lý do
Giữ 2 KafkaTemplate riêng	notificationTopic vẫn dùng String, order-placed-topic dùng typed JSON — không breaking change
ADD_TYPE_INFO_HEADERS = false	Consumer (product-service) không cần phụ thuộc vào package path của OrderPlacedEvent từ producer
Key = orderId	Kafka dùng key để phân vùng → tất cả event của cùng đơn hàng đi vào cùng partition → đảm bảo thứ tự
@Qualifier	Phân biệt rõ 2 bean KafkaTemplate khi inject

Bước 3: Tái cấu trúc logic lỗi (Refactoring OrderService) Nhóm cung cấp đoạn mã nguồn cũ của class OrderService và yêu cầu AI thực hiện một Single Task: Xóa bỏ ProductClient (gọi đồng bộ) và thay thế bằng việc gọi OrderEventProducer sau khi lưu Database.

Hình minh họa: Prompt cung cấp Old Code và yêu cầu tái cấu trúc

Tiếp theo, tôi cung cấp cho bạn logic của class OrderService hiện tại (tôi sẽ dán nó ở dưới).
TASK: Hãy refactor lại hàm placeOrder(). Xóa bỏ đoạn code dùng ProductClient (FeignClient) để gọi đồng bộ sang product-service. Thay vào đó, sau khi lưu đơn hàng vào DB thành công, hãy gọi OrderEventProducer.sendOrderPlacedEvent(...).

```
OLD_CODE: {{{
@Service
@RequiredArgsConstructor
public class OrderService {

    private final OrderRepo orderRepo;
```

Đáp lại prompt này, AI đã xử lý cực kỳ xuất sắc khi không chỉ cung cấp mã nguồn mới mà còn đưa ra bảng "Diff so sánh cũ -> mới", chỉ rõ dòng code nào là Tight Coupling đã bị xóa bỏ, và vẽ lại luồng sơ đồ kiến trúc thay đổi từ Synchronous sang Event-driven.

Kết quả Refactor

Diff so sánh cũ → mới

```
java

// ✖ TRƯỚC: Tight Coupling – gọi Feign TRƯỚC khi save DB
for (ProductDTO product : payload.getItems()) {
    try {
        productClient.decreaseQuantity(product.getId(), product.getQuantity()); // sync call
    } catch (Exception e) {
        throw new ResponseStatusException(BAD_REQUEST, ...); // nếu product-service chết → order fail
    }
}
Order savedOrder = orderRepo.save(orderEntity); // save sau

// ✔ SAU: Event-Driven – save DB TRƯỚC, publish event SAU
Order savedOrder = orderRepo.save(orderMapper.toEntity(payload)); // ☺ DB first
Payload savedPayload = orderMapper.toDTO(savedOrder);

for (ProductDTO product : savedPayload.getItems()) { // ☺ async events
    orderEventProducer.publishOrderPlaced(orderId, product.getId(), product.getQuantity());
}
```

Thay đổi kiến trúc

Thay đổi kiến trúc

TRƯỚC (synchronous):

order-service —Feign→ product-service (nếu product-service chết → order fail)

|
└─Kafka→ notification-service

SAU (event-driven):

order-service —Kafka→ order-placed-topic → product-service (decoupled)

|
└─Kafka→ notificationTopic → notification-service

Bước 4: Xây dựng Consumer an toàn (Resilient Consumer) Chuyển sang product-service, nhóm tiếp tục prompt AI tạo @KafkaListener để hứng sự kiện. Để kiểm tra tư duy xử lý lỗi của AI, nhóm đặt ra một ràng buộc: *"Nếu số lượng tồn kho không đủ để trừ, hãy in ra một dòng log cảnh báo (không ném Exception làm chết Consumer)"*. Nhờ đó, AI đã sinh ra mã nguồn an toàn, đồng thời giải thích rõ hệ quả: nếu throw Exception, Kafka sẽ rơi vào vòng lặp retry liên tục (infinite loop) gây tốn 100% CPU, trong khi việc log warning giúp hệ thống giữ được sự ổn định (resilient).

Hình minh họa: Luồng hoạt động hoàn chỉnh và lý giải cách xử lý lỗi tại Consumer

Bây giờ chuyển sang product-service.

CONTEXT: product-service quản lý entity Product (có trường id và quantity).

TASK:

Tạo class OrderEventConsumer.

Viết một hàm sử dụng annotation @KafkaListener để lắng nghe từ order-placed-topic.

Trong hàm này, nhận OrderPlacedEvent dưới dạng JSON, tìm Product trong database theo productId, trừ đi số lượng (quantity) tương ứng và lưu lại vào DB.

CONSTRAINTS:

Nếu số lượng tồn kho không đủ để trừ, hãy in ra một dòng log cảnh báo (không ném Exception làm chết Consumer).

Cung cấp file cấu hình application.yml cơ bản cho Kafka Consumer.



Tổng kết implementation

Files đã tạo



```
product-service/
├─ event/
│   └─ OrderPlacedEvent.java    ← NEW: Kafka payload (tách biệt với order-service)
├─ config/
│   └─ KafkaConsumerConfig.java ← NEW: Consumer factory với JsonDeserializer
├─ consumer/
│   └─ OrderEventConsumer.java  ← NEW: @KafkaListener handler
└─ resources/
    └─ application.properties   ← UPDATED: Kafka consumer config
```

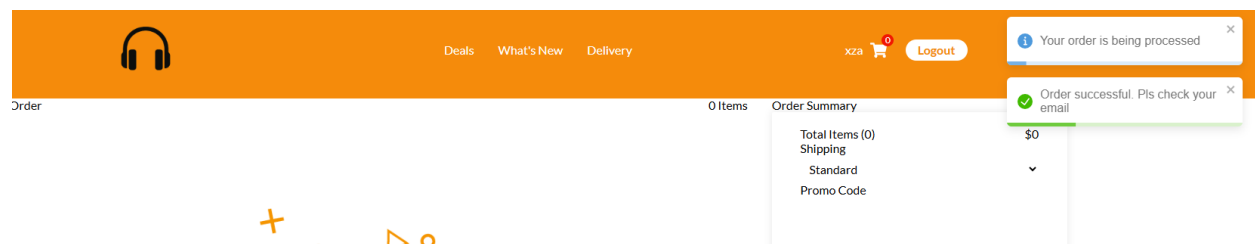
Flow hoàn chỉnh sau refactor

```
[Client] POST /order
|
▼
order-service
├─ save Order → MongoDB ✅
├─ publish OrderPlacedEvent → "order-placed-topic" (per item)
├─ publish notification → "notificationTopic"
|
▼
product-service Consumer
├─ findById(productId)
├─ quantity < requested? → ⚠️ LOG WARNING, skip (không crash)
└─ quantity >= requested → save MongoDB ✅
```

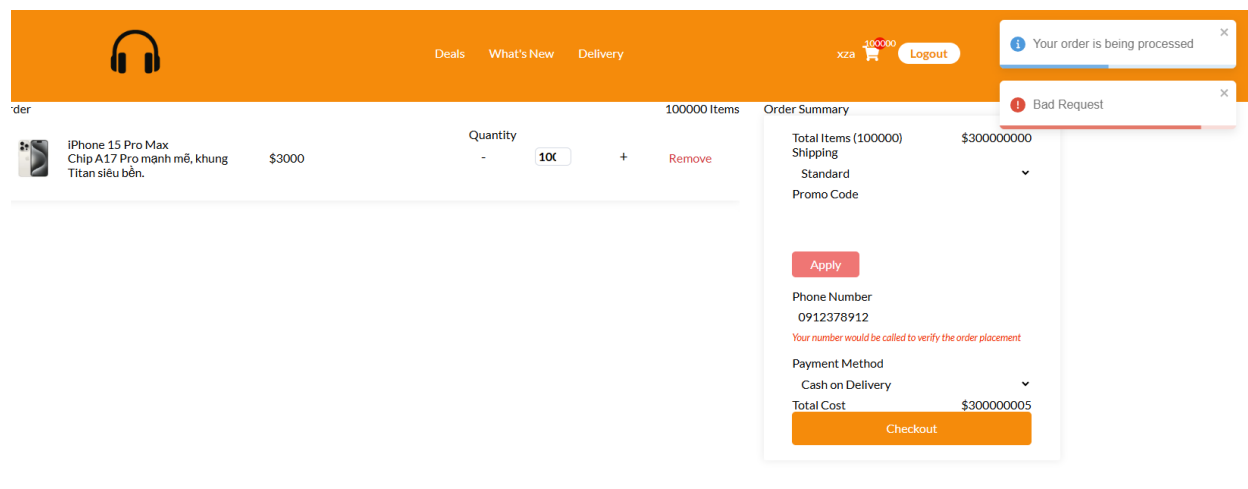
Tại sao không throw Exception trong Consumer?

Hành vi	Hệ quả
Throw exception	Kafka retry message liên tục → infinite loop → CPU 100%
Log warning + return	Consumer tiếp tục xử lý message tiếp theo → system ổn định
Catch-all + log error	Message lỗi format → bỏ qua, không crash → resilient

Khi số lượng đặt < tồn kho:



Hình ảnh minh họa khi đặt số lượng vượt quá tồn kho



3. Kết quả đạt được: Bằng chuỗi prompt bài bản, nhóm đã gỡ bỏ hoàn toàn sự phụ thuộc đồng bộ trong luồng đặt hàng chính. Hiện tại, order-service phản hồi ngay lập tức cho người dùng sau khi lưu dữ liệu vào cơ sở dữ liệu và đẩy thông điệp vào Kafka. Các tác vụ trừ tồn kho và gửi thông báo được xử lý ngầm (background), giúp cải thiện đáng kể tốc độ phản hồi (Latency) và nâng cao khả năng chịu tải của hệ thống.

2.3. Tích hợp cơ chế tự phục hồi (Circuit Breaker) với Resilience4j

1. Vấn đề hiện tại: Như đã phân tích tại Bài 07, hệ thống đang thiếu hụt cơ chế tự phục hồi (Resilience). Các lời gọi đồng bộ (REST API) giữa các dịch vụ không có phương án dự phòng. Khi product-service gặp sự cố (bị sập hoặc phản hồi chậm), các yêu cầu từ order-service sẽ bị treo (timeout), dẫn đến cạn kiệt tài nguyên (Thread exhaustion) và gây ra hiệu ứng lỗi dây chuyền (Cascading Failure) toàn hệ thống.

2. Quá trình ứng dụng GenAI để xây dựng lá chắn bảo vệ: Để khắc phục điểm yếu này, nhóm đã quyết định triển khai mô hình Circuit Breaker thông qua thư viện Resilience4j của Spring Cloud. Quá trình tích hợp được thực hiện bài bản thông qua việc thiết kế các Prompt có tính ràng buộc cao:

Bước 1: Thiết lập Dependency và Cấu hình Hạ tầng (Infrastructure Configuration)

Nhóm không tự mò mẫm cấu hình mà yêu cầu AI cung cấp chính xác đoạn mã XML cho pom.xml và cấu hình YAML cho application.yml. Đặc biệt, nhóm đã sử dụng **Constraint Prompting** để ép AI phải tuân thủ các thông số ngắt mạch do nhóm thiết kế (sliding-window-size: 10, failure-rate-threshold: 50%, wait-duration: 10s) thay vì để AI tự sinh các giá trị mặc định.

Hình minh họa: Prompt yêu cầu cấu hình Resilience4j với các ràng buộc khắt khe

CONTEXT: Dự án Spring Boot Microservices của tôi có order-service giao tiếp đồng bộ với product-service thông qua Spring Cloud OpenFeign.

TASK:

Cung cấp đoạn mã XML để thêm dependency Resilience4j và AOP (dùng cho annotation) vào file pom.xml của order-service.

Viết cấu hình cho Resilience4j trong file application.yml của order-service.

CONSTRAINTS:

Cấu hình một instance có tên là productServiceCircuitBreaker.

Sliding window size: 10 requests.

Tỷ lệ lỗi cho phép (Failure rate threshold): 50%.

Thời gian chờ khi mạch mở (Wait duration in open state): 10 giây.

Chỉ cung cấp mã cấu hình, không giải thích dài dòng.

Kết quả, AI đã cung cấp chính xác các dependency cần thiết (bao gồm cả spring-boot-starter-aop bắt buộc để Annotation hoạt động) và file YAML cấu hình hoàn chỉnh.

```
xml

<!-- Resilience4j Circuit Breaker -->
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>

<!-- AOP - bắt buộc để @CircuitBreaker annotation hoạt động -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

application.yml — cấu hình instance productServiceCircuitBreaker :

```
yml

resilience4j:
  circuitbreaker:
    instances:
      productServiceCircuitBreaker:
        sliding-window-size: 10      # 10 requests
        failure-rate-threshold: 50   # 50% lỗi → mở mạch
        wait-duration-in-open-state: 10s # chờ 10s
        permitted-number-of-calls-in-half-open-state: 3
        minimum-number-of-calls: 5
        automatic-transition-from-open-to-half-open-enabled: true
```

Bước 2: Tích hợp Annotation và Xây dựng Kịch bản Dự phòng (Fallback Method) Sau khi cấu hình xong, nhóm tiếp tục yêu cầu AI áp dụng Circuit Breaker vào mã nguồn Java. Nhóm đã cung cấp Context là phương thức kiểm tra sản phẩm qua FeignClient và yêu cầu AI sử dụng Annotation @CircuitBreaker. Đồng thời, AI bị buộc phải viết một hàm dự phòng fallbackGetProduct trả về một ngoại lệ tùy chỉnh (ServiceUnavailableException) với thông báo thân thiện thay vì ném ra lỗi HTTP 500 mặc định.

Tiếp theo, chúng ta sẽ áp dụng cấu hình này vào project.

CONTEXT: Trong order-service, tôi có class ProductClient (được annotate bằng @FeignClient) và class OrderService gọi productClient.getProductById(productId) để kiểm tra sản phẩm.

TASK:

Hãy viết lại đoạn code của class OrderService để áp dụng Circuit Breaker.

CONSTRAINTS:

Sử dụng annotation @CircuitBreaker(name = "productServiceCircuitBreaker", fallbackMethod = "fallbackGetProduct") bọc quanh hàm gọi sang product-service.

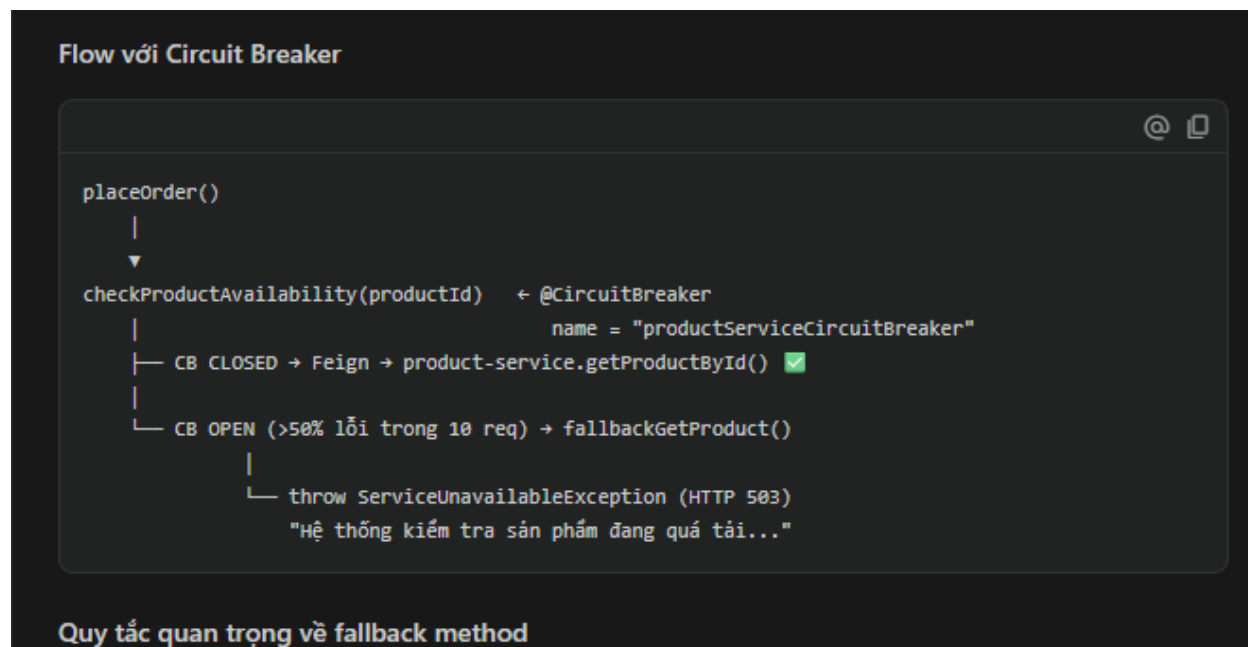
Viết hàm fallbackGetProduct. Hàm này sẽ được gọi khi product-service bị sập hoặc quá tải.



Files đã tạo/sửa

```
order-service/
├── exception/
│   └── ServiceUnavailableException.java  ← NEW
├── config/
│   └── ProductClient.java              ← UPDATED: thêm getProductById
├── service/
│   └── OrderService.java               ← UPDATED: CB + fallback
└── resources/
    └── application.yml                 ← NEW: Resilience4j config
```


AI đã thực thi xuất sắc yêu cầu này và còn cung cấp thêm một sơ đồ "Flow với Circuit Breaker" mô tả rõ ràng sự chuyển đổi trạng thái giữa CLOSED (gọi API bình thường) và OPEN (ném lỗi dự phòng khi tỷ lệ lỗi vượt 50%).



3. Kết quả đạt được: Nhờ việc ứng dụng GenAI để thiết lập Circuit Breaker, hệ thống giờ đây đã có khả năng tự "ngắt mạch" để bảo vệ chính nó. Khi kiểm thử thực tế bằng cách chủ động tắt product-service (Fault Injection), người dùng nhấn đặt hàng không còn phải đợi Timeout hay nhận lỗi "Bad Request" khó hiểu như trước.

Thay vào đó, hệ thống lập tức phản hồi thông báo thân thiện "*Hệ thống kiểm tra sản phẩm đang quá tải...*", đồng thời bảo toàn tài nguyên cho các yêu cầu khác. Điều này đã nâng cao đáng kể chỉ số sẵn sàng và độ tin cậy của toàn bộ nền tảng thương mại điện tử.

2.4. Tái cấu trúc mã nguồn (Refactoring Code Quality)

1. Vấn đề hiện tại: Qua rà soát mã nguồn ở Bài 07, hệ thống đang bộc lộ hai vấn đề lớn về "Code Smell":

- **Sự tồn tại của các đoạn mã lặp (Logic Duplication):** Các Controller hiện tại đang phải xử lý quá nhiều khối try-catch thủ công để định dạng dữ liệu trả về khi có lỗi. Việc này gây khó khăn cho bảo trì và dẫn đến sự thiếu đồng nhất trong Response format.
- **Vi phạm nguyên lý thiết kế SOLID:** Cụ thể là vi phạm nguyên lý Đơn nhiệm (Single Responsibility Principle - SRP). Điển hình như class `OrderServiceImpl`, nó đang đóng

vai trò là một "God Class" khi vừa chứa logic nghiệp vụ, vừa ôm đồm việc chuyển đổi (mapping) dữ liệu thủ công từ DTO sang Entity.

2. Quá trình ứng dụng GenAI để Tái cấu trúc: Để xử lý triệt để hai vấn đề trên, nhóm đã sử dụng phương pháp Prompt Engineering nhằm tách nhỏ các nhiệm vụ cho AI, buộc AI tuân thủ nguyên tắc Clean Code thay vì chỉ viết thêm code mới.

- **Bước 1: Giải quyết mã lộn xộn bằng Global Exception Handler** Đầu tiên, nhóm yêu cầu AI thiết kế một class `ApiResponse` chuẩn hóa, sử dụng thư viện Lombok để giảm boilerplate code. AI đã chủ động thêm annotation `@JsonFormat` cho trường `timestamp` và giải thích rõ ràng lý do: *"...nếu không có, Jackson mặc định serialize LocalDateTime thành mảng số [2024,5,1...] thay vì chuỗi string thân thiện với frontend."*

Hình minh họa: AI tạo cấu trúc dữ liệu lỗi và giải thích việc sử dụng `@JsonFormat`

CONTEXT: Trong dự án Spring Boot Microservices, các API đang trả về định dạng lỗi không đồng nhất.
TASK: Viết một class `ApiResponse` để chuẩn hóa dữ liệu trả về khi có lỗi.
CONSTRAINTS:

Class có các trường: `timestamp` (`LocalDateTime`), `status` (`int`), `error` (`String`), `message` (`String`), và `path` (`String`).

Sử dụng thư viện Lombok (`@Data`, `@Builder`) để giảm boilerplate code.

Worked for 27s >

Xong. Một lưu ý quan trọng về `@JsonFormat` :

Tại sao cần `@JsonFormat` ? Nếu không có, Jackson mặc định serialize `LocalDateTime` thành mảng số `[2024,5,1,10,30,0]` thay vì string `"2024-05-01 10:30:00"` — không thân thiện với frontend.

Cách dùng trong `GlobalExceptionHandler` :

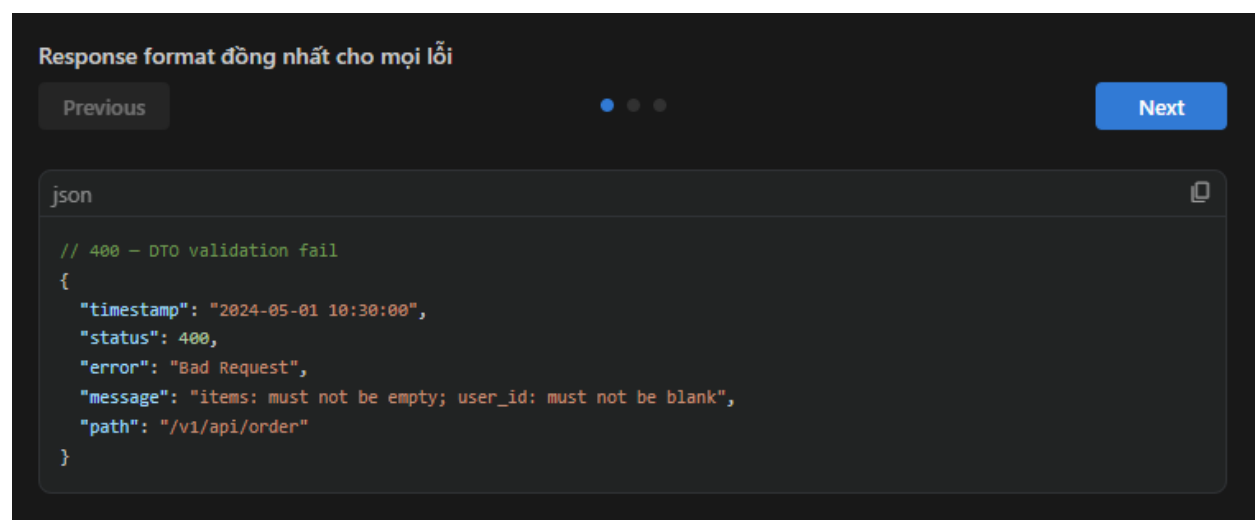
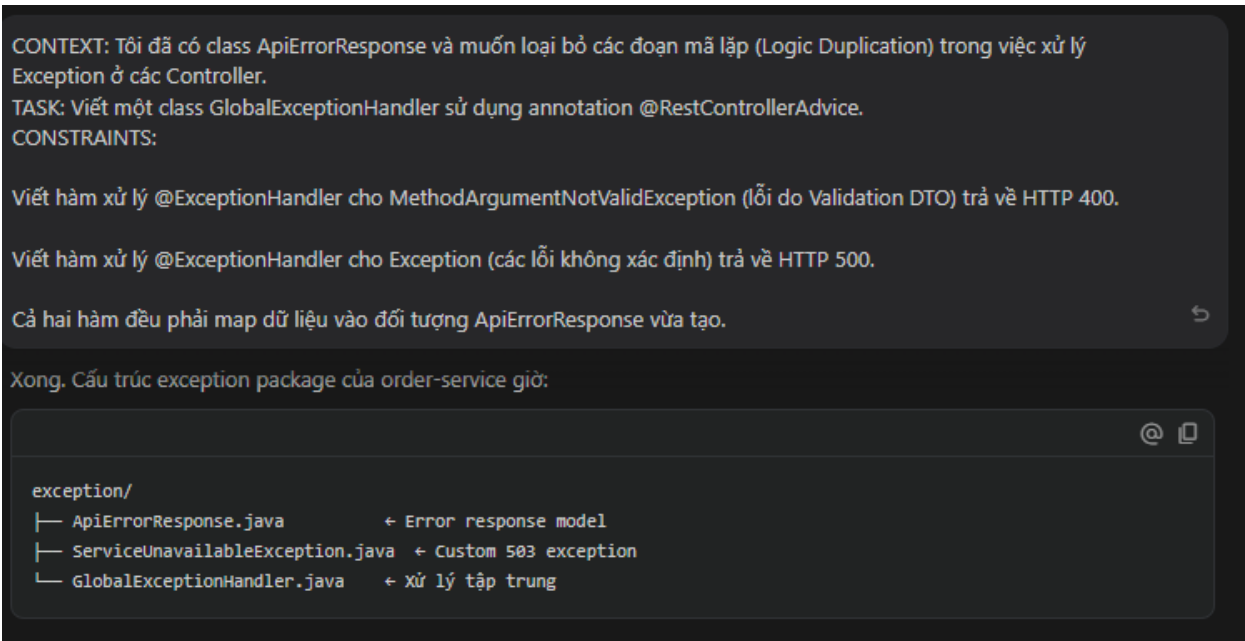
```
java
@ExceptionHandler(ServiceUnavailableException.class)
public ResponseEntity<ApiResponse> handleServiceUnavailable(
    ServiceUnavailableException ex, HttpServletRequest request) {

    ApiResponse body = ApiResponse.builder()
        .timestamp(LocalDateTime.now())
        .status(503)
        .error("Service Unavailable")
        .message(ex.getMessage())
        .path(request.getRequestURI())
        .build();

    return ResponseEntity.status(503).body(body);
}
```

Tiếp theo, nhóm cấp cho AI nhiệm vụ tạo GlobalExceptionHandler với annotation @RestControllerAdvice. AI đã sinh ra các phương thức xử lý lỗi tập trung (như lỗi Validation 400 và lỗi Server 500) và ánh xạ chúng vào đối tượng ApiErrorResponse vừa tạo, đảm bảo một định dạng JSON duy nhất cho toàn hệ thống.

Hình minh họa: Prompt tạo Global Exception Handler và các định dạng JSON lỗi đồng nhất



Response format đồng nhất cho mọi lỗi

Previous

Next

json

```
// 503 - Circuit Breaker mở (product-service sập)
{
  "timestamp": "2024-05-01 10:30:00",
  "status": 503,
  "error": "Service Unavailable",
  "message": "Hệ thống kiểm tra sản phẩm đang quá tải...",
  "path": "/v1/api/order"
}
```

Response format đồng nhất cho mọi lỗi

Previous

Next

json

```
// 500 - Lỗi không xác định
{
  "timestamp": "2024-05-01 10:30:00",
  "status": 500,
  "error": "Internal Server Error",
  "message": "An unexpected error occurred. Please contact support.",
  "path": "/v1/api/order"
}
```

Bước 2: Phá vỡ "God Class" bằng cách bóc tách Logic Mapping Để giải quyết vi phạm SRP, nhóm yêu cầu AI tạo một class OrderMapper riêng biệt. Ràng buộc đưa ra là phải tự viết code ánh xạ rõ ràng thay vì dùng thư viện bên ngoài (như MapStruct) để dễ kiểm soát.

CONTEXT: Trong dự án, OrderServiceImpl đang vi phạm nguyên lý Single Responsibility (SRP) vì nó tự thực hiện việc chuyển đổi thủ công từ OrderRequest (DTO) sang Order (Entity).

TASK: Hãy tạo một class OrderMapper được annotate bằng @Component.

CONSTRAINTS:

Class có hàm toEntity(OrderRequest request) trả về đối tượng Order.

Class có hàm toResponse(Order entity) trả về đối tượng OrderResponse.

Đảm bảo code rõ ràng, không sử dụng các thư viện mapping bên ngoài như MapStruct lúc này để dễ kiểm soát. Chỉ trả về mã nguồn.

Worked for 1m >

```
dto/
├─ OrderRequest.java    ← Input từ client (có @Valid constraints, không có id/status)
├─ OrderResponse.java   ← Output ra client (có id, status, createdAt)
└─ Payload.java         ← Giữ nguyên (legacy)

mapper/
├─ OrderMapper.java     ← 4 methods: toEntity(Payload), toDTO(Order),
                           toEntity(OrderRequest), toResponse(Order)
```

```

/**
 * Chuyển OrderRequest (DTO từ client) sang Order (Entity lưu DB).
 * id và createdAt KHÔNG map từ request – DB/service tự sinh.
 * status được set mặc định là "PENDING" tại tầng Service.
 */
public Order toEntity(OrderRequest request) {
    return Order.builder()
        .items(request.getItems())
        .totalItemCount(request.getTotalItemCount())
        .delivery_type(request.getDelivery_type())
        .delivery_type_cost(request.getDelivery_type_cost())
        .cost_before_delivery_rate(request.getCost_before_delivery_rate())
        .cost_after_delivery_rate(request.getCost_after_delivery_rate())
        .promo_code(request.getPromo_code())
        .contact_number(request.getContact_number())
        .user_id(request.getUser_id())
        .paymentMethod(request.getPaymentMethod())
        .build();
}

/**
 * Chuyển Order (Entity từ DB) sang OrderResponse (DTO trả về client).
 * Chỉ expose các field cần thiết, không lộ internal data.
 */
public OrderResponse toResponse(Order order) {
    return OrderResponse.builder()
        .id(order.getId())
        .items(order.getItems())
        .totalItemCount(order.getTotalItemCount())
        .delivery_type(order.getDelivery_type())
        .delivery_type_cost(order.getDelivery_type_cost())
        .cost_before_delivery_rate(order.getCost_before_delivery_rate())
        .cost_after_delivery_rate(order.getCost_after_delivery_rate())
        .promo_code(order.getPromo_code())
        .contact_number(order.getContact_number())
        .user_id(order.getUser_id())
        .paymentMethod(order.getPaymentMethod())
        .status(order.getStatus())
}

```

Sau khi có OrderMapper, nhóm đưa đoạn code cũ (Old Code) của OrderServiceImpl cho AI và yêu cầu refactor. Kết quả vô cùng ấn tượng: đoạn code cũ (TRƯỚC) đầy rẫy các thao tác set/get thủ công và trung gian thừa thãi đã được AI dọn dẹp sạch sẽ. Đoạn code mới (SAU) chỉ còn giữ lại đúng 5 bước nghiệp vụ cốt lõi, gọi qua Mapper để xử lý việc chuyển đổi dữ liệu.

CONTEXT: Bây giờ tôi sẽ cung cấp cho bạn class OrderService cũ, class này đang chứa các đoạn code chuyển đổi dữ liệu lộn xộn.

TASK: Hãy refactor lại class OrderService này.

CONSTRAINTS:

Inject (Tiêm phụ thuộc) class OrderMapper vừa tạo vào thông qua Constructor.

Thay thế toàn bộ các đoạn code gán dữ liệu thủ công (vd: order.setProductId(...)) bằng việc gọi hàm từ OrderMapper.

```
// ❌ TRƯỚC – Payload làm mọi thứ, logic lẫn lộn với data conversion
public Payload placeOrder(Payload payload) {
    payload.setStatus("PENDING"); // mutation trực tiếp DTO
    for (ProductDTO item : payload.getItems()) { ... }
    Order savedOrder = orderRepo.save(orderMapper.toEntity(payload));
    Payload savedPayload = orderMapper.toDTO(savedOrder); // trung gian thừa
    String orderId = savedPayload.getId();
    for (ProductDTO product : savedPayload.getItems()) { ... }
    ...
    return savedPayload;
}

// ✅ SAU – 5 bước rõ ràng, OrderMapper xử lý toàn bộ conversion
public OrderResponse placeOrder(OrderRequest request) {
    request.getItems().forEach(item ->
        checkProductAvailability(item.getId())); // ⚡ validate
    Order orderEntity = orderMapper.toEntity(request); // ⚡ map request
    // → entity
    orderEntity.setStatus("PENDING");
    Order savedOrder = orderRepo.save(orderEntity); // ⚡ lưu DB
    savedOrder.getItems().forEach(item -> // ⚡ publish
        Kafka
            orderEventProducer.publishOrderPlaced(...));
    return orderMapper.toResponse(savedOrder); // ⚡ map entity
    // → response
}
```

3. Kết quả đạt được: Nhờ áp dụng GenAI với các ràng buộc thiết kế chặt chẽ, mã nguồn của hệ thống đã trở nên tinh gọn và dễ bảo trì hơn hẳn. Việc quản lý lỗi tập trung giúp chuẩn

hóa giao tiếp với phía Frontend, trong khi việc bóc tách OrderMapper đã trả lại sự trong sáng (clean) cho các class xử lý nghiệp vụ, đảm bảo tuân thủ nguyên lý thiết kế phân lớp và SOLID.

2.5. Tự động hóa kiểm thử (Unit Testing)

1. Vấn đề hiện tại: Như đã đề cập trong Bài 07, hệ thống đang gặp thách thức lớn về "Độ bao phủ mã nguồn" (Code Coverage). Việc thiếu hụt các bản kiểm thử đơn vị (Unit Tests) khiến cho mỗi lần bảo trì hay nâng cấp tính năng mới (như thêm Kafka, Circuit Breaker) đều tiềm ẩn rủi ro phá vỡ các chức năng hiện tại. Tuy nhiên, việc tự viết Unit Test thủ công cho các hệ thống phụ thuộc nhiều vào Database và Message Broker thường mất rất nhiều thời gian.

2. Quá trình ứng dụng GenAI để Tự động hóa kiểm thử:

Để khắc phục điểm yếu này một cách nhanh chóng và hiệu quả, nhóm đã tận dụng GitHub Copilot Chat để tự động sinh ra các kịch bản kiểm thử cho OrderServiceImpl. Thay vì để AI sinh test một cách ngẫu nhiên, nhóm đã cung cấp các ràng buộc rõ ràng để định hướng việc giả lập (Mocking).

- **Bước 1: Khởi tạo Unit Test cơ bản (Happy Path) với Mockito**

Nhóm cung cấp mã nguồn của OrderServiceImpl và yêu cầu AI sử dụng JUnit 5 cùng Mockito. Ràng buộc quan trọng nhất là yêu cầu AI phải sử dụng hàm verify() của Mockito để đảm bảo rằng khi một đơn hàng được tạo thành công, hàm save() của Repository và hàm send() của Kafka Producer phải được gọi chính xác 1 lần.

Hình minh họa: Prompt yêu cầu viết Unit Test cho luồng Happy Path

```
CONTEXT: Trong dự án Spring Boot, tôi có class OrderService phụ thuộc vào
OrderRepository (MongoDB) và OrderEventProducer (Kafka).
TASK: Viết class OrderServiceImplTest để kiểm thử đơn vị (Unit Test).
CONSTRAINTS:

Sử dụng JUnit 5 và Mockito (@ExtendWith(MockitoExtension.class)).

Sử dụng @Mock cho các phụ thuộc và @InjectMocks cho OrderServiceImpl.

Viết một test case 'Happy Path' (ví dụ: testPlaceOrder_Success) để kiểm tra
luồng đặt hàng thành công.

Sử dụng verify() của Mockito để đảm bảo hàm save() của Repository và
send() của Producer đã được gọi đúng 1 lần
```

AI đã sinh ra cấu trúc test tuân thủ chặt chẽ mô hình AAA (Arrange - Act - Assert), đồng thời bổ sung thêm bước Verify để xác minh tương tác với các hệ thống phụ thuộc.

Cấu trúc test

```
// Arrange – chuẩn bị dữ liệu + stub behavior
when(productClient.getProductById("prod-1")).thenReturn(ResponseEntity.ok(product));
when(orderMapper.toEntity(request)).thenReturn(mappedEntity);
when(orderRepo.save(any())).thenReturn(savedOrder);

// Act – gọi method cần test
OrderResponse result = orderService.placeOrder(request);

// Assert – kiểm tra output
assertEquals("order-xyz-123", result.getId());

// Verify – kiểm tra interaction với dependency
verify(orderRepo, times(1)).save(any(Order.class));
verify(orderEventProducer, times(1)).publishOrderPlaced("order-xyz-123", "prod-1", 2);
verifyNoMoreInteractions(orderEventProducer); ← không có side-effect ngoài ý muốn
```

Bước 2: Bổ sung các luồng lỗi (Edge Cases) để tăng Coverage Chỉ kiểm tra luồng thành công là chưa đủ để đạt độ bao phủ cao. Nhóm tiếp tục prompt AI sinh thêm các kịch bản ngoại lệ (Bad Path). Ví dụ: Giả lập đầu vào bị thiếu thông tin hoặc giả lập Database bị lỗi (MongoException). Đặc biệt đối với lỗi Database, nhóm đặt ràng buộc AI phải dùng verify(...,

never()) để khẳng định rằng Kafka Producer tuyệt đối không được phát đi thông báo nếu dữ liệu chưa được lưu thành công.

Hình minh họa: Prompt yêu cầu bổ sung các test cases xử lý lỗi

```
Tốt. Để tăng độ bao phủ mã nguồn (Code Coverage), tôi cần bổ sung thêm các kịch bản kiểm thử ngoại lệ.
CONTEXT: Class OrderServiceImplTest vừa tạo.
TASK: Viết thêm 2 test cases xử lý lỗi.
CONSTRAINTS:

Test case 1: testPlaceOrder_InvalidData - Giả lập đầu vào OrderRequest bị thiếu productId. Khẳng định (Assert) rằng hàm ném ra lỗi IllegalArgumentException.

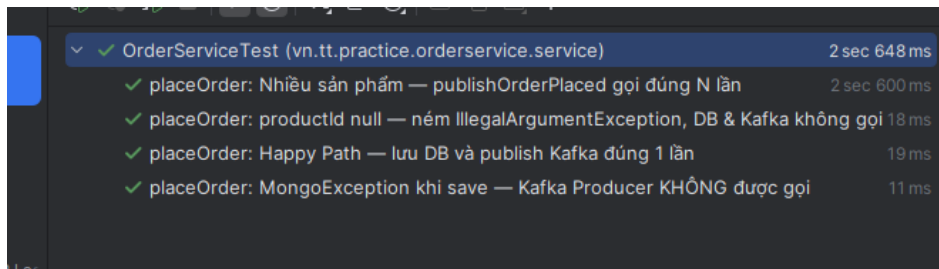
Test case 2: testPlaceOrder_DatabaseError - Giả lập OrderRepository.save() ném ra lỗi MongoException. Đảm bảo luồng xử lý lỗi (hoặc Fallback) hoạt động đúng và Kafka Producer KHÔNG được gọi (verify(..., never()).send(...)).
```

Nhờ các prompt chi tiết này, AI đã tự động xây dựng và tổng kết 4 kịch bản kiểm thử (Test Cases) bao quát mọi ngóc ngách của hàm placeOrder, từ luồng thành công cho đến các trường hợp dữ liệu lỗi hoặc hệ thống Database gặp sự cố.

Tổng kết 4 test cases		
Test	Mục tiêu	Verify chính
testPlaceOrder_Success	Happy path	save() ×1, publishOrderPlaced() ×1, sendOrderEvent() ×1
testPlaceOrder_MultipleItems	N sản phẩm	publishOrderPlaced() đúng N lần với đúng params
testPlaceOrder_InvalidData_NullProductId	productId = null	Throw IllegalArgumentException, save() never, Kafka never
testPlaceOrder_DatabaseError_KafkaNotCalled	DB timeout	Throw MongoException, save() ×1 (đã gọi nhưng fail), Kafka never

3. Kết quả đạt được: Chỉ với vài thao tác Prompt Engineering, nhóm đã tự động hóa việc viết mã kiểm thử phức tạp. Khi thực thi trên IDE, toàn bộ các test cases đều "Pass" (màu xanh). Việc này không chỉ nâng cao độ bao phủ mã nguồn một cách nhanh chóng mà còn tạo ra một "mạng lưới an toàn" (Safety Net), giúp nhóm tự tin hơn khi thực hiện các thay đổi kiến trúc lớn trong hệ thống.

Hình minh họa: Kết quả chạy Unit Test thành công trên IDE



Dưới đây là nội dung chi tiết cho **Phần 2.6** trong Báo cáo của bạn, tập trung vào việc chuẩn hóa cấu hình và thiết lập cơ chế truy vết nhật ký (Logging).

Bạn hãy copy đoạn văn bản này vào file Word/Docs và chèn các hình ảnh minh chứng tương ứng nhé.

2.6. Chuẩn hóa Cấu hình và Logging (Observability)

1. Vấn đề hiện tại: Hệ thống Microservices ở Bài 07 đang gặp hai vấn đề lớn cản trở việc triển khai và giám sát trên môi trường thực tế (Production):

- **Cấu hình bị Hardcode:** Các thông số quan trọng như địa chỉ kết nối MongoDB, Kafka, và Eureka Server đang được ghi cứng (hardcode) trong file application.yml. Điều này khiến việc đóng gói Docker và triển khai lên các môi trường khác nhau (Local, Staging, Production) trở nên khó khăn, buộc phải sửa code và build lại nhiều lần.
- **Thiếu khả năng truy vết luồng (Distributed Tracing):** Khi một request đi từ API Gateway xuống order-service rồi sang product-service, nếu xảy ra lỗi, lập trình viên không thể biết các dòng log ở các service khác nhau thuộc về cùng một luồng request nào, biến hệ thống thành một "hộp đen" (black box).

2. Quá trình ứng dụng GenAI để chuẩn hóa và giám sát: Để đưa hệ thống đạt chuẩn Cloud-Native, nhóm đã sử dụng GenAI để tái cấu trúc lại cách quản lý cấu hình và xây dựng bộ lọc truy vết.

- **Bước 1: Chuyển đổi cấu hình tĩnh sang biến môi trường (Environment Variables)**

Nhóm yêu cầu AI viết lại file application.yml với ràng buộc rõ ràng: phải sử dụng cú pháp biến môi trường của Spring Boot kèm giá trị mặc định (Ví dụ: `${MONGO_URI:mongodb://localhost:27017/orderdb}`).

Hình minh họa: Prompt yêu cầu chuyển đổi cấu hình và bảng Mapping do AI tổng hợp

CONTEXT: Trong dự án Microservices, file application.yml của các service (ví dụ: order-service) đang bị hardcode các thông tin kết nối quan trọng như địa chỉ MongoDB, Kafka và Eureka.

TASK: Viết lại cấu trúc file application.yml.

CONSTRAINTS:

Sử dụng cú pháp biến môi trường của Spring Boot kèm giá trị mặc định (Ví dụ: \${MONGO_URI:mongodb://localhost:27017/orderdb}).

Áp dụng cho các thành phần: spring.data.mongodb.uri, spring.kafka.bootstrap-servers, và eureka.client.serviceUrl.defaultZone

Worked for 44s >

Xong. Bảng mapping env vars:

Env Variable	Default (local dev)	Docker (production)
MONGO_URI	<code>mongodb://localhost:27017/orderdb</code>	<code>mongodb://mongo:27017/database</code>
KAFKA_BOOTSTRAP_SERVERS	<code>localhost:9092</code>	<code>kafka:9092</code>
EUREKA_URL	<code>http://localhost:8761/eureka/</code>	<code>http://eureka-server:8761/eureka/</code>
ZIPKIN_URL	<code>http://localhost:9411/api/v2/spans</code>	<code>http://zipkin:9411/api/v2/spans</code>
SERVER_PORT	<code>8083</code>	override tùy ý

Kết quả, AI không chỉ sinh ra mã nguồn YAML chuẩn xác mà còn chủ động lập một bảng "Mapping env vars" so sánh rõ ràng sự khác biệt giữa giá trị mặc định (chạy Local) và giá trị khi đưa lên Docker (Production).

Bước 2: Xây dựng cơ chế truy vết nhật ký tập trung với Correlation ID Để giải quyết bài toán "hộp đen" của log, nhóm thiết kế một prompt yêu cầu AI tạo TrackingFilter tại API Gateway. Các ràng buộc (Constraints) được đưa ra rất khắt khe về mặt kỹ thuật: AI phải tự động sinh UUID nếu header X-Correlation-ID chưa tồn tại, đưa ID này vào Mapped Diagnostic Context (MDC) của SLF4J, và truyền tiếp xuống các service phía sau.

Hình minh họa: Prompt yêu cầu tạo TrackingFilter và sơ đồ luồng hoạt động do AI phác thảo



Nhờ cách đặt vấn đề rõ ràng, AI đã sinh ra mã nguồn bộ lọc hoàn hảo. Đặc biệt, AI còn vẽ ra một sơ đồ "Flow hoạt động" dạng text, mô tả chi tiết đường đi của Request từ Client qua Gateway, cách ID được đưa vào log, và cách xóa ID khỏi MDC ở khối finally để tránh rò rỉ bộ nhớ (memory leak).

3. Kết quả đạt được: Thông qua việc ứng dụng GenAI có định hướng:

- Hệ thống hiện tại đã hoàn toàn tách biệt cấu hình khỏi mã nguồn (Configuration as Code). Việc triển khai lên Docker giờ đây chỉ cần truyền tham số qua file docker-compose.yml mà không cần sửa bất kỳ dòng code nào.
- Cơ chế Logging đã được chuẩn hóa. Mọi dòng log từ API Gateway đến các service nội bộ đều được gắn chung một mã định danh (Correlation ID). Khi có sự cố, kỹ sư chỉ cần lọc theo mã ID này là có thể nhìn thấy toàn bộ hành trình của request, giúp thời gian chẩn đoán và khắc phục lỗi giảm đi đáng kể.

Chương 3. KẾT LUẬN & ĐÁNH GIÁ

1. Hiệu quả của việc ứng dụng GenAI trong tái cấu trúc hệ thống Từ những điểm nghẽn kiến trúc và chất lượng mã nguồn được phân tích trong Bài 07 (bao gồm High Coupling, thiếu hụt cơ chế tự phục hồi, rủi ro bảo mật và độ bao phủ kiểm thử thấp), nhóm đã thực hiện thành công một chiến dịch tái cấu trúc toàn diện trong Bài 08. Điểm đột phá của đề án lần này không nằm ở việc lập trình thủ công, mà ở việc ứng dụng bài bản Trí tuệ Nhân tạo tạo sinh (GenAI) thông qua các công cụ như GitHub Copilot Chat.

Bằng việc áp dụng phương pháp Prompt Engineering nâng cao với bộ quy tắc 5S (Structured, Surrounding context, Single task, Specific, Short) kết hợp cùng kỹ thuật ép suy luận (Chain-of-Thought) và thiết lập ràng buộc (Constraint Prompting), nhóm đã xử lý được một khối lượng công việc đồ sộ trong thời gian rất ngắn. Các công nghệ cốt lõi của kiến trúc Microservices như: giao tiếp bất đồng bộ qua **Apache Kafka**, ngắt mạch bảo vệ hệ thống với **Resilience4j**, bảo mật tập trung bằng **JWT**, và tự động hóa kiểm thử bằng **JUnit 5 / Mockito** đều được tích hợp thành công, chuẩn xác và tuân thủ đúng nguyên lý thiết kế SOLID.

2. Đánh giá vai trò của Kỹ sư phần mềm trong kỷ nguyên AI Qua quá trình thực nghiệm "Vibe Coding" và điều hướng AI sinh mã nguồn, nhóm rút ra kết luận sâu sắc: **GenAI không thay thế kỹ sư phần mềm, mà đóng vai trò như một "Chuyên gia lập trình cặp" (Pair Programmer) siêu tốc.**

Chất lượng của mã nguồn do AI sinh ra phụ thuộc hoàn toàn vào tư duy kiến trúc (Architecture Mindset) của người đặt câu hỏi. Khi người kỹ sư không nắm rõ luồng hệ thống, AI dễ dàng rơi vào trạng thái "ảo giác" (hallucination) và sinh ra những đoạn code sai lệch. Ngược lại, khi người kỹ sư làm chủ được thiết kế, biết cách chia nhỏ bài toán (Single Task), cung cấp đúng ngữ cảnh (Context) và đặt ra các giới hạn kỹ thuật khắt khe (Constraints), AI

sẽ phát huy tối đa sức mạnh, giúp tiết kiệm đến 80% thời gian viết boilerplate code và tự động hóa kiểm thử.

3. Tầm nhìn và định hướng Thành công của dự án này chứng minh rằng việc tích hợp AI vào Vòng đời phát triển phần mềm (SDLC) là một xu thế tất yếu. Hệ thống Microservices thương mại điện tử của nhóm hiện tại không chỉ được gỡ bỏ các rủi ro vận hành mà còn đạt được trạng thái lý tưởng về độ lỏng lẻo (Loose Coupling), tính bền bỉ (Resilience) và sẵn sàng cho việc triển khai trên nền tảng điện toán đám mây (Cloud-Native).

Đây là tiền đề vững chắc để nhóm tiếp tục phát triển, mở rộng dự án và tự tin áp dụng AI vào các bài toán kỹ thuật phức tạp hơn trong môi trường doanh nghiệp thực tế sau này.